localAssetStorageStack Gaps

Blocking

1. **Pod addresses change every run.** `up.js` sets the Solid server's base URL to the loopback address plus a random port, so every restart changes the address of every pod, WebID and ACL. The lifecycle service also only accepts a `podUrl` on the `css:3000` origin, which doesn't match the addresses CSS actually hands out, and containers calling `css:3000` will likely be rejected for the wrong host. Fix: give CSS one fixed hostname and port used both inside and outside the containers (for example a `css.localhost` name, or a small proxy), and keep the random ports only for the host-side links.
2. **The lifecycle service never reads the pod.** `pin()` trusts the `content` or `hash` the registrar sends. Nothing fetches `podUrl` from CSS or checks its ETag, so a pinned hash proves nothing about what the pod holds. The service needs its own Solid client identity (DPoP) and has to read the pod back before it pins, and again before any binding.
3. **An agent can publish with no reviewer.** The `review` class allows `review → publish`, and the `agent` role can set `next: "publish"`. Reviewer dispositions are recorded but never gate a transition. Fix: entering `publish`, which carries `pod:write`, should require an `approve` disposition on the exact revision from a reviewer who isn't the author.
4. **Identity is five shared role tokens.** `reviewer` is free text, so one reviewer token can sign off under any name, which defeats the one-decision-per-reviewer rule. `admin` passes every role check, so the admin token can act as engine, agent and reviewer. CSS is already an OIDC provider, so use WebIDs as the principals and keep the tokens for services only.

Serious

5. **Access scopes aren't enforced.** `contextGrant.scopes` (`pod:write`, `rag:read`, `beacon:read`) are just labels. No token is issued and no ACL changes, and `beacon` isn't defined anywhere. CSS registration is also open to anyone (`identity/access/public.json`), and the shared `reality-engine` network means any OpenClaw or localAIStack container can create an account and write to pods directly, bypassing the lifecycle service.
6. **Receipts can't serve as outside evidence.** HMAC-SHA256 is symmetric, so anyone who can verify a receipt can also forge one. It's computed over `JSON.stringify`, which doesn't fix key order, so Lisp, C++ and Scala can't reproduce it, and verification uses a plain string comparison. Use Ed25519 over canonical JSON (RFC 8785).
7. **Storage isn't append-only.** Every write rewrites one `state.json` file, with no fsync, no lock and no event log. `Object.freeze` doesn't survive a reload, so 'immutable' proposals and receipts can be edited on disk. Revisions accept any string and duplicates, with no expected-previous-revision check, so two registrars can overwrite each other.
8. **Dependencies are stored but never used.** A new revision marks only that asset's own machines as stale; downstream assets aren't touched. Assets also have no entity or period.

Structural

9. **The review machine sits outside the engine.** `GUARDRAIL_CLASSES` is hard-coded JavaScript. It isn't an RE machine definition, isn't in the domain manifest and isn't covered by three-engine parity, so it's a second rules engine running beside the real one. Several bindings can also be open at once in the same state.

10. **Nothing connects dokieli to the lifecycle service.** Annotations made in dokieli land in the pod, but `/annotations` only accepts a reviewer-token POST. No inbox listener or adapter moves them into the findings.
11. **It doesn't match the IFRS document model.** The IFRS Accounting domain's `document_workflow` works by entity and period, with 68 asset types, DRAFT/IN_REVIEW/APPROVED/STALE, and `evidenceRefs` and `upstreamRefs`. This service has its own, different model. Pick one canonical lifecycle and treat this service as its Solid-backed repository.

Build and operations

12. **Nothing is pinned.** dokieli builds from `main` with `npm install`, and the images use floating tags (`community-server:7`, `node:22-alpine`, nginx `stable`). That conflicts with the suite's build-provenance practice.
13. **Only the lifecycle container is hardened.** It's the only one running read-only with capabilities dropped. CSS has no healthcheck, and `depends_on` doesn't wait for CSS to be ready.
14. **CI only tests the lifecycle service.** It never builds dokieli, never starts the stack and never touches CSS.
15. **No backup or export** for the `css-data` and `lifecycle-data` volumes.
16. **Housekeeping:** `.gitignore` carries leftover CUDA patterns (`*.ptx`, `*.cubin`).